UNIT 3:

5. A department maintains student roll numbers dynamically. Write a C program using a Singly Linked List to create the list, insert at the beginning and end, search for a specified roll number, delete a specified roll number, and display the updated list after each operation. Handle the case when a requested roll number is not available.

Source code :-

```
#include <stdio.h>

#include <stdlib.h>

struct Node

{

   int roll;

   struct Node *next;

};

struct Node *head = NULL;

void create()

{

   int n, roll, i;

   struct Node *newNode, *temp;

   printf("Enter number of students: ");

   scanf("%d", &n);

   for (i = 0; i < n; i++)

   {

      printf("Enter roll number: ");

      scanf("%d", &roll);

      newNode = malloc(sizeof(struct Node));

      newNode->roll = roll;

      newNode->next = NULL;

      if (head == NULL)

      {

         head = newNode;

      }

      else {
```

```
            temp = head;
            while (temp->next != NULL)
                temp = temp->next;
            temp->next = newNode;
        }
    }
}
void insertBeginning()
{
    int roll;
    struct Node *newNode;
    printf("Enter roll number: ");
    scanf("%d", &roll);
    newNode = malloc(sizeof(struct Node));
    newNode->roll = roll;
    newNode->next = head;
    head = newNode;
    printf("Inserted at beginning\n");
}
void insertEnd()
{
    int roll;
    struct Node *newNode, *temp;
    printf("Enter roll number: ");
    scanf("%d", &roll);
    newNode = malloc(sizeof(struct Node));
    newNode->roll = roll;
    newNode->next = NULL;
    if (head == NULL)
        head = newNode;
    else {
```

```
        temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
    }
    printf("Inserted at end\n");
}
void search()
{
    int roll, pos = 1;
    struct Node *temp = head;
    printf("Enter roll number to search: ");
    scanf("%d", &roll);
    while (temp != NULL)
    {
        if (temp->roll == roll)
        {
            printf("Roll number found at position %d\n", pos);
            return;
        }
        temp = temp->next;
        pos++;
    }
    printf("Roll number not found\n");
}
void deleteNode()
{
    int roll;
    struct Node *temp, *prev;
    printf("Enter roll number to delete: ");
    scanf("%d", &roll);
```

```
    temp = head;
    prev = NULL;
    while (temp != NULL && temp->roll != roll)
    {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL)
    {
        printf("Roll number not found\n");
        return;
    }
    if (prev == NULL)
        head = temp->next;
    else
        prev->next = temp->next;
    free(temp);
    printf("Roll number deleted\n");
}
void display()
{
    struct Node *temp = head;
    if (head == NULL)
    {
        printf("List is empty\n");
        return;
    }
    printf("Student Roll Numbers: ");

    while (temp != NULL)
```

```
    {
        printf("%d -> ", temp->roll);
        temp = temp->next;
    }
    printf("NULL\n");
}
int main()
{
    int choice;
    while (1)
    {
        printf("\n--- Singly Linked List ---\n");
        printf("1. Create\n");
        printf("2. Insert Beginning\n");
        printf("3. Insert End\n");
        printf("4. Search\n");
        printf("5. Delete\n");
        printf("6. Display\n");
        printf("7. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                create();
                display();
                break;
            case 2:
                insertBeginning();
                display();
                break;
```

```
            case 3:
                insertEnd();
                display();
                break;
            case 4:
                search();
                break;
            case 5:
                deleteNode();
                display();
                break;
            case 6:
                display();
                break;
            case 7:
                return 0;
            default:
                printf("Invalid choice\n");
        }
    }
    return 0;
}
```

Output :-

